

· KLDP.org · [KLDP Wiki](http://KLDP.Wiki) · KLDP.net · [통합 검색](#) ·

 

Subversion Book/Repository Administration



[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook/Preface](#) › [SubversionBook/Introduction](#) › [SubversionBook/GuidedTour](#) › [SubversionBook/BasicConcepts](#) › [SubversionBook/BranchingAndMerging](#) ›

SubversionBook/RepositoryAdministration

저장소 관리 Subversion의 저장소는 복수의 프로젝트를 위한 버전 관리된 데이터를 보관하는 중심적인 장소입니다. 이런 이유로 저장소는 관리자에게 있어서는 모든 애정과 관심을 쏟을 대상이 될지도 모릅니다. 하지만 저장소는 일반적으로는 그만큼 복잡한 관리가 필요하지 않습니다. 잠재적인 문제를 피하거나 실제로 일어나는 문제를 안전하게 해결하기 위해서는 어떻게 하면 올바르게 설정하고 관리할 수 있는지 이해하는 것이 중요합니다. 이 장에서는 Subversion의 저장소를 어떻게 만들고 설정하는지 그리고 저장소를 네트워크로 접근하기 위해서는 어떻게 하면 좋은지에 대해 논의합니다. svnlook과 svnadmin 도구(이 둘은 Subversion이 제공하는 툴입니다)의 이용 방법을 포함해서 저장소 관리에 대해서도 이야기할 것입니다. 자주 있는 질문과 실수를 언급하고 저장소에서 어떻게 데이터를 배치하는게 좋은지 도와줄 것입니다. 만약 Subversion 저장소의 버전 관리하에 있는 데이터에 사용자로서 접근할 뿐이라면 (즉 Subversion 클라이언트만 사용한다면) 이 장은 그냥 지나쳐도 됩니다. 그러나 Subversion 저장소를 관리하고 싶다면 이걸 꼭 말하면 매우 고상한 일처럼 생각됩니다만 작업 카피 너머에 있는 신비의 영역에 흥미를 가지는 모든 사람을 의미하는 말입니다. 이 장을 특별히 주의해서 보세요. 물론 관리할 저장소가 하나도 없으면 저장소 관리자가 될 수 없습니다. 저장소의 기초 트랜잭션(transaction)과 리비전의 이해 개념적으로 말하면 Subversion 저장소는 디렉토리 트리의 연속입니다. 각각의 트리는 저장소에 있는 파일이나 디렉토리가 특정 순간에 어떻게 보이는지에 대한 순간포착 사진(snapshot)입니다. 클라이언트의 조작에 의해 만들어지는 순간포착 사진을 리비전(revision)이라고 합니다. 각각의 리비전은 트랜잭션(transaction) 트리로 되어 있습니다. 커밋하면 클라이언트는 로컬에서의 변경 사항(과 클라이언트의 커밋 처리를 시작할 때 저장소에 일어나는 부가적인 변경 사항)을 반영한 Subversion 트랜잭션을 만들어 그 트리가 일련의 연속포착 사진 중 마지막이 되도록 저장소에 저장하라고 명령합니다. 커밋이 성공하면 트랜잭션은 새로운 리비전 트리로 승격되고 새로운 리비전 번호를 받습니다. 커밋이 어떤 이유로 실패하면 트랜잭션은 파괴되고 클라이언트는 실패했다는 통지를 받습니다. 갠신 처리도 이와 같이 동작합니다. 클라이언트는 작업 카피 상태를 반영한 임시 트랜잭션 트리를 만듭니다. 저장소는 그 트랜잭션 트리와 요청된 버전의 리비전 트리(보통은 최신의 혹은 가장 새로운 트리)와 비교해서 작업 카피를 그 리비전의 트리로 변형하려면 어떠한 변경이 필요한지 정보를 되돌려줍니다. 갠신이 완료된 후 임시 트랜잭션은 삭제됩니다. 트랜잭션 트리를 이용하는 것이 저장소에서 관리되는 파일 시스템을 변경하는 유일한 방법입니다. 그러나 트랜잭션의 생존 시간이 완전히 임의인 것을 이해하는 것이 중요합니다. 갠신의 경우 트랜잭션은 곧바로 소멸하는 일시적인 트리입니다. 커밋의 경우는 트랜잭션은 영구적인 리비전으로 바뀝니다(커밋이 실패했을 때는 삭제되지만). ऐसा 버그가 있으면 트랜잭션은 저장소에 남겨져 버릴지도 모릅니다(그러나 이것은 디스크 용량을 차지할 뿐 나쁜 영향을 주지는 않습니다). 이론적으로 언젠가는 통합된 작업 환경을 지원하는 어플리케이션은 트랜잭션의 생존 기간을 좀 더 유연하게 관리할 수 있게 될지도 모릅니다. 클라이언트가 저장소에 대한 수정 내용의 기술을 끝낸 후에도 리비전이 될 후보 트랜잭션이 정지한 상태에 머무르는 시스템을 생각할 수도 있습니다. 이것은 새로운 커밋을 다른 사람 예를 들어 관리자나 엔지니어링 QA팀이 재검토하는 것을 가능하게 해서 그 트랜잭션을 리비전으로 승격시키거나 철회하거나 할 수가 있게 되겠지만, 저장소를 관리할 때 반드시 하지 않으면 안 되는 것은 무엇일까요? 대답은 간단합니다. Subversion 저장소를 관리한다면 저장소의 상태를 살피는 일의 일부로서 리비전과 트랜잭션을 조사해야 한다는 것입니다. 버전화 되지 않는 속성 Subversion 저장소에서 트랜잭션과 리비전은 그것에 딸린 속성을 가질 수가 있습니다. 그러한 속성은 일반적인 키-값의 대응으로 관련한 트리에 대한 정보를 보관하는데 이용됩니다. 속성의 이름과 값은 나머지 트리 데이터와 함께 저장소의 파일 시스템에 보관됩니다. 리비전과 트랜잭션의 속성은 작업 카피에 의해 관리될 수 없는 정보들 같이 파일이나 디렉토리에 강하게 관련되지 않은 트리의 정보를 기억해 두는 데 편리합니다. 예를 들어 새로운 커밋 트랜잭션이 저장소에 만들어지면 Subversion은 그 트랜잭션에 svn:date라는 속성을 추가합니다. 트랜잭션이 만들어진 시각을 나타내는 표시입니다. 커밋이 완료되어 트랜잭션이 영구적인 리비전이 될 때 트리에는 리비전 작성자의 이름(svn:author)과 리비전에 붙여진 로그 메시지(svn:log)라는 속성이 추가됩니다. 리비전과 트랜잭션의 속성은 버전화 되지 않는 속성입니다. 속성은 수정되면 그 이전의 값을 영구히 잃게 됩니다. 마찬가지로 리비전 트리 자신은 불변입니다만 트리의 속성은 그렇지 않습니다. 언제라도 리비전 속성을 추가, 삭제, 수정 할 수 있습니다. 새로운 리비전을 커밋한 후에 잘못된 정보가 있거나 로그 메시지에 오타가 있는 것을 알았을 때에는 단지 svn:log 속성의 값을 올바른 로그 메시지로 바꾸기만 하면 됩니다. 저장소 작성과 설정 Subversion 저장소를 작성하는 일은 매우 간단한 작업입니다. svnadmin 유틸리티에 저장소를 만드는 서브 커맨드가 있습니다. 새 저장소를 만드려면 단지 다음과 같은 명령만 쓰면 됩니다. \$ svnadmin create path/to/repos 이것으로 path/to/repos 디렉토리에 새 저장소가 만들어집니다. 새 저장소는 리비전 0으로 태어납니다. 리비전 번호 0번에는 최상위 루트(/) 파일 시스템 디렉토리만 있습니다. 초기 상태에서 리비전 0은 리비전 속성을 하나만 가지고 있으며, 그것은 svn:date입니다. 이 속성은 저장소가 만들어진 시간을 나타냅니다. svnadmin의 인수로 전달되는 경로는 일반적인 파일시스템 경로이지 svn 클라이언트 프로그램이 저장소를 참조할 때 사용하는 URL이 아님을 주의하세요. svnadmin과 svnlook도 서버측의 유틸리티라고 생각하세요. 이 둘은 저장소가 설치된 컴퓨터에서 저장소를 조사하거나 상태를 변경하는데 사용되고, 네트워크를 통해서 작업을 수행할 수는 없습니다. Subversion 초심자가 자주 하는 실수는 이 프로토콜들에 URL을 사용하는 것입니다. (혹은 "로컬" URL이라는 의미로 file:이라고 지정해 버리는 것입니다.) svnadmin create 명령을 실행하면 지정된 디렉토리에는 반짝반짝 빛나는 새 Subversion 저장소가 생깁니다. 그 디렉토리에 실제로 무엇이 생겼는지 들여다 봅시다. \$ ls repos dav/ db/ format hooks/ locks/ README.txt README.txt 파일을 제외하면 저장소 디렉토리는 하위 디렉토리의 모임입니다. Subversion의 일반적인 설계 사상과 같이 모듈화에 많이 신경을 쓴 것입니다. 계층적으로 구조화 된 것이 어지럽게 흩어져 있는 상태보다 바람직합니다. 새 저장소 디렉토리에 있는 항목들에 대해서 간단히 설명하겠습니다. dav Apache와 mod_dav_svn의 관리용 데이터 디렉토리입니다.db Subversion 파일시스템 데이터 저장소를 구성하는 DB 테이블 전체 즉, 메인 Berkeley DB 환경이 있습니다. (여기에 모든 버전화

Login:

Page 10 of 10

Password:

Copyright © 2010 Pearson Education, Inc.

Join

OK

Recent investments will yield a slight profit.

KLDPCCodeFest/UnicodeTranslation
UnderConstruction
엄길웅
Spirit



kss@kldp.org

된 데이터가 보관됩니다) format 저장소 레이아웃의 버전 번호를 나타내는 정수값 하나가 들어있는 파일입니다. hooks 혹은 스크립트 템플릿(template) 전체(와 설치된 혹은 스크립트)가 보관되는 디렉토리입니다. locks 저장소에 접근한 사람을 기록하는데 사용되는 잠금 데이터를 위한 디렉토리입니다. README.txt Subversion 저장소를 보는 사람을 위한 정보가 담긴 파일입니다. 일반적으로 직접 저장소를 건드려서는 안 됩니다. svnadmin 도구는 저장소를 변경하는데 충분한 기능을 제공하며, 저장소를 구성하는 일부분을 조정하는 데에는 (Berkeley DB의 도구 한 벌 따위의) 서드파티 도구를 사용할 수 있습니다. 그래도 몇몇 예외가 있으며 차차 설명하겠습니다. 혹은 스크립트(Hook Scripts) 혹은(hook)은 새 리비전이 생성되거나 버전화되지 않은 속성이 수정되는 일(event)이 일어났을 때 실행되는 프로그램입니다. 각각의 혹은 어떤 일이 일어났는지, 작업 대상은 무엇인지, 그 일을 한 사람은 누구인지 알 수 있는 충분한 정보를 전달 받습니다. 혹은 출력이나 반환값에 따라 혹은 프로그램은 처리를 계속하거나 종료하거나 몇 가지 방법으로 일시 중단하거나 합니다. 기본값에 따르면 hooks 디렉토리에는 다양한 저장소 혹은 템플릿이 있습니다. \$ls repos/hooks/ post-commit.tmpl pre-revprop-change.tmpl post-revprop-change.tmpl start-commit.tmpl pre-commit.tmpl Subversion이 가지고 있는 혹은에는 각각 하나의 템플릿이 있고, 템플릿 스크립트의 내용을 보면 그 스크립트가 어떤 트리거(trigger)를 실행하는지, 어떤 데이터가 그 스크립트에 전달되는지 알 수 있습니다. 또 많은 템플릿들에는 유용한 작업을 수행하기 위해서 Subversion이 제공하는 다른 프로그램들과 혹은 스크립트를 어떻게 결합해서 쓰는지 보여주는 예제가 있습니다. 실제로 혹은 설치하는 것은 repos/hooks 디렉토리에 혹은 이름으로 (start-commit이나 post-commit 같이) 실행 프로그램이나 스크립트를 두기만 하면 됩니다. 유닉스(Unix)에서는 이 말의 의미는 스크립트나 프로그램(셸 스크립트, 파이썬 프로그램, 컴파일된 C 실행파일 등)을 정확하게 혹은 이름과 똑같은 이름으로 가져다 놓으라는 말입니다. 물론 템플릿 파일들이 정보를 제공하기 위한 목적만으로 제공되는 것은 아닙니다. 유닉스에서 혹은 설치하는 가장 간단한 방법은 템플릿 파일을 .tmpl 확장자를 떼 새 파일로 복사해서 내용을 고치고 그 파일에 실행권한을 주는 것입니다. 그러나 윈도우즈(Windows)에서는 실행가능한 파일임을 나타내기 위해서 특수한 확장자를 사용하기 때문에, 실행 프로그램에는 혹은 이름 뒤에 .exe나 .com를 붙이고 일괄작업(batch) 파일에는 .bat를 붙여야 합니다. 현재 Subversion 저장소에는 다섯 종류의 혹은이 구현되어 있습니다. start-commit 커밋 트랜잭션이 만들어지기 전에 실행됩니다. 전형적으로 사용자에게 커밋 권한이 있는지 없는지 결정하는데 사용됩니다. 저장소는 이 프로그램에 두 개의 인수를 건네주는데, 그것은 저장소 경로와 커밋하려는 사용자 이름입니다. 만약 프로그램이 0이 아닌 값을 돌려주었을 경우 트랜잭션이 만들어지기 전에 커밋이 중지됩니다. pre-commit 트랜잭션의 완결 후 실제로 커밋이 일어나기 전에 실행됩니다. 전형적으로는 허용되지 않은 내용이나 장소로 커밋하는 것(예를 들면, 특정 브랜치에 커밋하려면 버그 추적 시스템(bug tracker)이 주는 표가 있어야 하는 사이트도 있고, 로그 메시지가 꼭 있어야 하는 사이트도 있습니다)을 막는데 쓰입니다. 저장소는 이 프로그램에 두 개의 인수를 건네주는데, 그것은 저장소의 경로와 커밋될 트랜잭션의 이름입니다. 만약 이 프로그램이 0이 아닌 값을 돌려주었을 경우 커밋은 취소되고 트랜잭션은 삭제됩니다. Subversion 배포본은 세세한 접근 제어를 실현할 수 있도록 pre-commit이 호출하는 접근 제어 스크립트들(Subversion 소스 트리의 tools/hook-scripts 디렉토리에 있습니다)을 포함하고 있습니다. 현시점에서는 이것이 httpd.conf가 제공하는 설정 이외에 한층 더 세세한 접근 제어를 실현하는 유일한 방법입니다. 향후의 버전에서는 파일시스템에 접근 제어 목록(ACL)을 직접 구현할 계획입니다. post-commit 이것은 트랜잭션이 커밋되어 새로운 리비전이 만들어진 후에 실행됩니다. 대부분의 사람들은 커밋에 대한 메일을 보내거나 저장소를 백업하는데 이 혹은을 사용합니다. 저장소는 이 프로그램에 두 개의 인수를 건네주는데, 저장소의 경로와 이번에 만들어진 새 리비전의 번호입니다. 이 프로그램의 종료 코드는 무시됩니다. Subversion 배포판은 commit-email.pl 스크립트를 포함하고 있습니다(Subversion 소스 트리의 tools/hook-scripts/ 디렉토리에 있습니다). 이것을 이번 커밋 대한 설명을 메일로 보내기 위해서 (혹은 로그 파일에 추가하기 위해서) 하기 위해서 사용할 수 있습니다. 메일의 내용은 변경된 경로의 목록, 커밋에 붙인 로그 메시지, 커밋한 사람, 커밋한 시각, 커밋의 변경 부분의 GNU diff 형식 표시입니다. Subversion이 제공하는 도구 중에 유용한 또 다른 도구는 hot-backup.py 스크립트입니다(Subversion 소스 트리의 tools/backup/ 디렉토리에 있습니다). 이 스크립트는 Subversion 저장소의 온라인 백업 (BerkeleyDB 데이터베이스에서 지원되는 기능입니다)을 실행합니다. 이 스크립트는 매 커밋마다 보관용 혹은 비상 복구용 저장소 스냅샷을 만드는데 사용할 수 있습니다. pre-revprop-change Subversion의 리비전 속성은 버전화되지 않기 때문에 속성(예를 들어 커밋 메시지 속성인 svn:log)에 대한 수정은 이전 속성값을 영원히 덮어써 버립니다. 이런 점 때문에 데이터를 잃어버릴 수 있으므로, Subversion은 필요에 따라 속성에 대한 변경을 저장소 관리자가 기록으로 남길 수 있도록 이 혹은(그리고 이것과 짝을 이루는 post-revprop-change)을 제공합니다. 이 혹은은 저장소에 그러한 변경이 발생하기 직전에 실행됩니다. 저장소는 이 혹은에 네 개의 인수를 전달하는데, 그것은 저장소 경로, 수정될 속성이 소속된 리비전, 변경하려는 사람의 인증된 사용자 이름, 속성의 이름입니다. post-revprop-change 앞서 말한 것 처럼 이 혹은은 pre-revprop-change 혹은과 쌍을 이룹니다. 사실 이 스크립트는 pre-revprop-change 혹은이 존재하지 않으면 실행되지 않습니다. 혹은 둘 다 존재하는 경우 post-revprop-change 혹은은 리비전의 속성이 변경된 직후에 실행됩니다. 전형적으로는 변경된 속성의 새 값을 메일로 알리는데 사용됩니다. 저장소는 네 개의 인수를 이 혹은에 전해주는데, 그것은 저장소 경로, 속성이 소속된 리비전 번호, 변경을 하려는 사람의 인증된 사용자 이름, 속성의 이름입니다. Subversion 배포판에는 propchange-email.pl 스크립트가 포함되어 있습니다(tools/hook-scripts/ 디렉토리에 있습니다). 리비전 속성 변경에 대한 정보를 메일로 보내기 위해서(혹은 로그 파일에 추가하기 위해서) 그것을 사용할 수 있습니다. 그 메일의 내용은 리비전과 변경된 속성 이름, 변경한 사람 이름, 속성의 새 값을 포함합니다. Subversion은 저장소에 접근하는 프로세스의 소유자로서 혹은을 실행하려고 합니다. 대부분의 경우 사용자는 아파치(Apache) HTTP 서버와 mod_dav_svn을 통해서 저장소에 접근하므로, 혹은을 실행하는 사용자와 같습니다. 혹은을 실행하려는 사용자가 실행할 수 있도록 OS 수준에서 혹은을 실행 권한을 주어야 합니다. 또 그 말은 혹은이 직간접적으로 접근하는 모든 파일이나 프로그램(Subversion 저장소도 포함하여)에 같은 사용자로 접근한다는 뜻입니다. 혹은이 지정된 작업을 수행하는데 권한 설정에 관련된 문제가 일어날 가능성이 있으므로 주의하시기 바랍니다. 버클리 DB 설정 버클리(Berkeley) DB 환경에는 한 번에 몇 개의 잠금(lock)이 허용되는지, 저널링 로그 파일의 크기 제한은 얼마큼인지 등에 대한 기본 설정값이 있습니다. 거기에 더해서 Subversion의 파일 시스템 코드가 설정해놓은 Berkeley DB 설정의 기본값이 있습니다. 그러나 특징적인 데이터나 접근 패턴을 가지는 저장소가 있을 것이고, 그런 저장소는 다른 설정값을 가지는 것이 바람직합니다. Sleepycat의 사람들(버클리 DB의 제작자들)은 다른 데이터베이스는 다른 요구를 처리해야 한다는 것을 알고있기 때문에, 실행시에 버클리 DB의 설정값을 바꿀 수 있는 방법을 제공해왔습니다. 버클리 DB는 환경 디렉토리에 DB_CONFIG 파일이 있는지 확인하고, 그 파일에 있는 옵션을 사용합니다. 당신의 저장소의 버클리 DB 설정 파일은 db 라는 환경 디렉토리 안의 repos/db/DB_CONFIG 파일입니다. Subversion이 저장소를 만들 때 이 파일도 만들어집니다. 이 파일에는 기본 설정과 함께 버클리 DB의 온라인 문서가 어디있는지에 대한 정보도 있으므로, 어느 옵션이 어떤 기능을 하는지에 대해 읽어볼 수 있습니다. 물론 지원되는 어떤 옵션이

라도 DB_CONFIG에 추가할 수 있습니다. Subversion은 그 파일의 내용을 읽거나 해석하지도 않고 그 파일에 설정된 값을 전혀 사용하지도 않지만, Subversion의 나머지 코드가 예측할 수 없는 방식으로 버클리 DB가 작동할 수 있는 설정은 피해주세요. DB_CONFIG에 대한 변경은 데이터베이스 환경을 복구할 때까지 (svnadmin recover 를 사용하여)는 효력을 발생하지 않습니다. 저장소의 유지보수 관리자용 도구 모음 svnlook svnlook은 저장소에 있는 다양한 리비전과 트랜잭션을 조사하는데 사용되는 도구입니다. 이 프로그램은 읽기 전용 도구이므로 저장소를 전혀 변경하지 않습니다. 전형적으로 svnlook은 커밋될 변경 사항을 보고하거나(pre-commit 후) 방금 커밋된 것이 무엇인지 보고하는(post-commit 후) 후에 의해서 사용됩니다. 저장소 관리자는 저장소 진단을 위해서 이 툴을 사용할 것입니다. svnlook은 사용법이 단순합니다. \$ svnlook help general usage: svnlook SUBCOMMAND REPOS_PATH [ARGS OPTIONS ...] Note: any subcommand which takes the '--revision' and '--transaction' options will, if invoked without one of those options, act on the repository's youngest revision. Type "svnlook help subcommand" for help on a specific subcommand. svnlook의 하위 명령어 대부분은 리비전이나 트랜잭션 트리를 다루며, 트리 자체에 대한 정보를 보여주거나 저장소의 예전 리비전들과 어떻게 다른지 보여줍니다. 리비전이나 트랜잭션을 지정하기 위해서 --revision과 --transaction 옵션을 사용하십시오. 리비전 번호는 자연수로 표시하지만 트랜잭션 이름은 알파벳과 숫자로 구성된 문자열로 표시한다는 것을 주의하세요. 파일시스템은 커밋되지 않은 트랜잭션(아직 새 리비전으로 바뀌지 않은 트랜잭션)만을 표시할 수 있다는 것을 기억하세요. 아마 대부분의 저장소에는 그런 트랜잭션이 없을 것입니다. 보통 트랜잭션은 커밋되거나 커밋이 중지된 후 지워지기 때문입니다. --revision이나 --transaction 옵션을 둘 다 지정하지 않으면 svnlook은 최신의 (혹은 HEAD) 리비전을 조사합니다. 따라서 /path/to/repos에 있는 저장소의 최신 리비전이 19일 때 아래에 있는 두 명령은 완벽히 똑같습니다. \$ svnlook info /path/to/repos \$ svnlook info /path/to/repos --revision 19 이 규칙에 대한 유일한 예외는 svnlook youngest입니다. 이것은 아무 옵션도 취하지 않고, HEAD 리비전의 번호를 표시할 뿐입니다. \$ svnlook youngest /path/to/repos 19 svnlook의 출력은 사람이 읽을 수도 있고 컴퓨터가 자동으로 처리할 수도 있도록 설계되었습니다. 하위 명령어 info를 봅시다. \$ svnlook info path/to/repos sally 2002-11-04 09:29:13 -0600 (Mon, 04 Nov 2002) 27 Added the usual Greek tree. info 하위 명령어의 출력은 다음과 같이 정의되어 있습니다. 작업자, 줄바꿈 날짜, 줄바꿈 로그 메시지의 길이, 줄바꿈 로그 메시지, 줄바꿈 이 출력은 인간이 읽을 수가 있습니다. 일자의 타임스탬프 등은 무엇인가 바이너리 표현과 같은 것이 아니고 텍스트 형식이 되어 있습니다. 그러나 이것은 또 머신도 해석할 수 있는 형식의 것입니다 로그 메시지는 복수행에 걸쳐 줄 수 있어 길이의 제한이 없기 때문에 svnlook은 메시지 자신의 앞에 그 길이를 표시합니다. 이것으로 이 커맨드의 스크립트나 다른 래퍼 프로그램은 영리한 판단을 할 수 있게 됩니다. 예를 들어 메시지에 얼마나의 메모리를 할당하면 좋은지 라든지 이벤트중에서 적어도 몇 바이트 스킵 해도 데이터 스트림이 끝나게 되지 않는지 등을 아는 것이 할 수 있습니다. 자주 있는 다른 svnlook 의 사용법은 리비전 또는 트랜잭션(transaction) 트리의 실제의 내용을 보는 것입니다. svnlook tree 커맨드의 출력을 조사해 지정한 트리 내부의 디렉토리나 파일을 표시시키면(그리고 옵션으로서 각각의 패스의 파일 시스템 리비전 ID를 표시시키면) 관리자는 죽어 버린 것처럼 보이는 트랜잭션(transaction)를 안전에 지을 수 있을까 제발을 판단할 수가 있습니다. 또 Subversion 개발자도 파일 시스템에 관련한 문제의 진단을 할 수가 있습니다. \$ svnlook tree path/to/repos --show-ids / 0. 0.1 A/ 2. 0.1 B/ 4. 0.1 lambda 5. 0.1 E/ 6. 0.1 alpha 7. 0.1 beta 8. 0.1 F/ 9. 0.1 mu 3. 0.1 C/ a. 0.1 D/ b. 0.1 gamma c. 0.1 G/ d. 0.1 pi e. 0.1 rho f. 0.1 tau g. 0.1 H/ h. 0.1 chi i. 0.1 omega k. 0.1 psi j. 0.1 iota 1. 0.1 svnlook은 그 밖에도 앞에서 살펴본 정보의 일부를 표시하거나, 지정한 리비전이나 트랜잭션에서 어느 경로가 수정되었는지를 보고하거나, 파일이나 디렉토리에 대한 텍스트나 속성의 차이점을 표시하거나 하는 등의 여러가지 문의를 할 수 있습니다. 아래에 svnlook이 제공하는 하위 명령어의 목록이 있습니다. author 그 트리를 만든 사람입니다. cat 트리에 있는 특정의 파일의 내용을 표시합니다. date 트리의 날짜입니다. changed 그 트리에서 변경이 있었던 파일과 디렉토리 목록을 보여줍니다. diff 변경된 파일의 unified diff를 표시합니다. dirs-changed 자신이 변경되었거나 하위의 파일이 변경된 디렉토리의 목록을 보여줍니다. history 버전화 된 경로(변경이나 복사가 일어난 곳)의 히스토리중에서 흥미로운 지점을 표시합니다. info 트리의 작성자, 날짜, 로그 메시지의 문자 수, 로그 메시지를 표시합니다. log 트리의 로그 메시지를 표시합니다. proplist 트리에 있는 경로에 대해서 설정된 속성의 이름과 값을 표시합니다. tree 트리의 목록을 표시합니다. 옵션으로 각각의 경로에 결합된 파일시스템 노드 리비전의 ID를 표시합니다. youngest 최신 리비전 번호를 표시합니다. svnadmin svnadmin은 저장소 관리자가 가장 자주 이용하는 프로그램입니다. Subversion 저장소를 작성하는 것 외에도 이 프로그램은 저장소를 유지보수 하는데 쓰이는 다양한 조작을 할 수 있습니다. svnadmin의 사용법은 svnlook과 비슷합니다. \$ svnadmin help general usage: svnadmin SUBCOMMAND REPOS_PATH [ARGS OPTIONS ...] Type "svnadmin help subcommand" for help on a specific subcommand. Available subcommands: create dump help (?, h) 이미 앞에서 svnadmin의 하위 명령어 create를 보았습니다(>참조). 다른 하위 명령어의 대부분을 이 장에서 설명합니다. 일단 이용 가능한 하위 명령어 전체를 살펴봅시다. create Subversion 저장소를 만듭니다. dump 저장소에서 지정 범위의 리비전의 내용을 portable 덤프 형식으로 덤프합니다. list-dblogs 저장소에 관계된 버클리 DB 로그 파일의 경로 목록을 보여줍니다. 이 목록은 모든 로그 파일을 포함합니다. 현재 Subversion이 이용하고 있는 것뿐만 아니라 더이상 이용하지 않는 것도 포함해서 말이지요. list-unused-dblogs 저장소에 관계된 버클리 DB 로그 파일 중에서 더이상 사용되지 않는 로그 파일의 경로 목록을 보여줍니다. 이런 로그 파일들은 저장소 레이아웃으로부터 안전하게 삭제할 수가 있지만 저장소를 복구할 때 필요할 것에 대비해 보관할 필요가 있을 수도 있습니다. load 하위 명령어 dump에서 생성된 것과 같은 portable 덤프 형식의 데이터 스트림으로부터 저장소로 리비전의 모임을 불러들입니다. lstxns 현재 저장소에 존재하고 있는 커밋되지 않은 Subversion 트랜잭션의 이름 목록을 보여줍니다. recover 필요에 따라서 저장소의 복구 조치를 실행합니다. 일반적으로 프로세스가 저장소와의 통신을 정상적으로 종료하지 못하고 오류가 발생한 후에 실행하게 됩니다. rmtxns 저장소에서 Subversion 트랜잭션을 깨끗이 삭제합니다(하위 명령어 lstxns의 출력을 이 프로그램에 입력하면 편리합니다). setlog 저장소에서 지정 리비전의 svn:log(커밋 로그 메시지) 속성의 값을 새로운 값으로 바꿉니다. verify 저장소의 내용을 검증합니다. 검증 절차에는 저장소에 있는 버전화된 데이터의 체크섬 비교도 포함됩니다. svnshell.py Subversion 소스 트리에는 저장소에 대한 헬 비슷한 인터페이스도 있습니다. svnshell.py Python 스크립트(소스 트리의 tools/examples/에 있습니다)는 저장소와 파일 시스템 프로그램 라이브러리에 접속하기 위해서 Subversion의 언어 제휴(language bindings, 이것을 동작시키려면 컴파일과 인스톨을 적절히 할 필요가 있습니다만)를 사용합니다. 실행하면 이 프로그램은 헬 프로그램과 같이 동작하며, 저장소의 다양한 디렉토리를 살펴볼 수 있습니다. 초기 상태에서는 저장소 HEAD 리비전의 루트 디렉토리에 위치하며, 이 상태로 명령 prompt가 표시됩니다. 언제라도 help 명령으로 사용 가능한 명령어 목록을 표시할 수 있습니다. \$ svnshell.py /path/to/repos rev: 2 /\$ help Available commands: cat FILE : FILE의 내용을 보여줍니다. cd DIR : DIR이라는 디렉토리로 이동합니다. exit : 셸을 종료합니다. ls [PATH] : 현재 디렉토리에 포함된 파일 목록을 보여줍니다. lstxns : 내

용을 볼 수 있는 트랜잭션의 목록을 보여줍니다. setrev REV : 살펴볼 리비전을 설정합니다. settxn TXN : 살펴볼 트랜잭션을 설정합니다. youngest : 살펴볼 수 있는 가장 최신 리비전 번호를 보여줍니다. rev: 2 /\$ 저장소의 디렉토리 구조를 돌아다니는 방법은 보통 Unix나 Windows 셸에서와 같이 cd명령어를 사용합니다. 명령 prompt는 항상 지금 보고 있는 것이 어느 리비전(prefixed by rev:)인지 또는 어느 트랜잭션(prefixed by txn:)인지, 또 그 안에서 어느 경로에 와있는지를 표시합니다. 현재의 리비전이나 트랜잭션은 각각setrev 와 settxn명령어로 변경할 수 있습니다. Unix 셸과 같이 현재 디렉토리의 내용을 표시하는데 ls 명령어를, 파일 내용을 보는데cat명령을 사용할 수 있습니다. svnshell을 이용하여 저장소 둘러보기 rev: 2 /\$ ls REV AUTHOR NODE-REV-ID SIZE DATE NAME

```
1 sally 2.0. 1 Nov 15 11:50 A/ 2 harry 1.0. 2 56 Nov 19 08:19 iota rev: 2 /$ cd A rev: 2 /A$ ls REV
AUTHOR NODE-REV-ID SIZE DATE NAME
```

```
1 sally 4.0. 1 Nov 15 11:50 B/ 1 sally a. 0.1 Nov 15 11:50 C/ 1 sally b. 0.1 Nov 15 11:50 D/ 1 sally
3.0. 1 23 Nov 15 11:50 mu rev: 2 /A$ cd D/G rev: 2 /A/D/G$ ls REV AUTHOR NODE-REV-ID SIZE
DATE NAME
```

1 sally e. 0.1 23 Nov 15 11:50 pi 1 sally f. 0.1 24 Nov 15 11:50 rho 1 sally g. 0.1 24 Nov 15 11:50 tau rev: 2 /A\$ cd ../. rev: 2 /\$ cat iota This is the file 'iota'. Added this text in revision 2. rev: 2 /\$ setrev 1; cat iota This is the file 'iota'. rev: 1 /\$ exit \$ 예제에서 볼 수 있는 것처럼 복수의 명령어를 세미콜론으로 구분하여 한 줄에 쓸 수 있습니다. 또 svnshell은 상대 경로와 절대 경로의 개념을 이해할 수 있으므로 ". " ". " 같이 특별한 디렉토리 이름도 이해할 수 있습니다. youngest 명령어는 최신 리비전 번호를 표시합니다. 이것은 setrev 명령어로 지정하는 인수의 리비전 범위를 아는데 사용할 수 있습니다. 0(영)부터 youngest 사이의 모든 리비전(리비전 번호는 정수인 것을 기억하세요)을 열람할 수가 있습니다. 열람 가능한 트랜잭션의 범위를 아는데는 간단한 방법이 없습니다. lstxns 명령을 이용하여 열람할 수 있는 트랜잭션의 목록을 조사하세요. 이 목록은 svnadmin lstxns 명령이 보여주는 것과 같은 목록이고, svnlook의 --transaction 옵션에 사용할 수 있는 트랜잭션들의 목록이기도 합니다. 셸의 이용을 종료하려면 exit 명령어를 사용합니다. 또는 Control-D(Win32 Python 패키지에서는 Windows 표준인 Control-Z를 사용합니다)를 눌러 EOF(파일 끝)문자를 보내도 됩니다. Berkeley DB 유틸리티 현재 Subversion 저장소는 데이터베이스 백엔드로 Berkeley DB를 사용합니다. 모든 파일 시스템 구조와 데이터는 저장소의 db라는 디렉토리 안에 테이블들로 보관됩니다. 이 디렉토리는 일반적인 Berkeley DB 환경 디렉토리이므로 Berkeley 데이터베이스 도구를 자유롭게 사용할 수 있습니다(SleepyCat 의 웹 사이트 <http://www.sleepycat.com/> 에서 이러한 도구의 문서를 볼 수가 있습니다). 일상적인 Subversion 작업에서는 이러한 도구는 불필요합니다. 그러나 Subversion이 자체적으로 제공하지 않는 중요한 기능이 몇 가지 있습니다. 예를 들어 Subversion은 Berkeley DB의 로그 기능을 사용하고 있으므로 데이터베이스는 일단 지금부터 하려고 하는 변경에 관한 내용을 로그에 쓴 후 실제 변경을 실행합니다. 이 방식은 무엇인가 잘 되지 않았을 때에 데이터베이스 시스템이 직전의 체크 포인트(시스템에 이상이 없는 지점)로 돌아갈 수 있도록 보장하고 데이터를 이용할 수 있는 상태가 될 때까지 트랜잭션을 다시 실행할 수 있도록 합니다. 이것은 subversion이 데이터베이스로서 Berkeley DB를 사용하는 주된 이유입니다. 시간이 지남에 따라 로그 파일들이 계속 쌓입니다. 이것은 데이터베이스 시스템의 기능 중에 하나입니다. 로그 파일만을 사용해서 데이터베이스 전체를 다시 만들 수가 있으므로, 로그 파일은 괴멸적인 데이터베이스 파괴를 복구하는데 매우 중요합니다. 그러나 대개 디스크를 공간을 확보하기 위해서 Berkeley DB 가 이용하고 있지 않은 로그 파일을 다른 곳에 보관하고 그 후 디스크로부터 삭제하고 싶다고 생각할 것입니다. Berkeley DB는 특정한 데이터베이스와 연관된 로그 파일과 더이상 이용하지 않는 로그 파일의 목록을 표시하는 db_archive 유틸리티를 를 제공합니다. 이 방법으로 어느 파일을 보관하고 지워도 되는지 알 수 있습니다. svnadmin 도구는 이 Berkeley DB 도구의 편리한 래퍼(wrapper)를 제공하고 있습니다. \$ svnadmin list-unused-dblogs /path/to/repos /path/to/repos/log. 00000000031 /path/to/repos/\$. 00000000032 /path/to/repos/log. 00000000033 \$ svnadmin list-unused-dblogs /path/to/repos | xargs rm ## disk space reclaimed! Subversion의 저장소는 post-commit 후 스크립트를 사용합니다. post-commit 혹은 저장소의 온라인 백업(hot backup)을 실행한 후 불필요한 로그 파일을 삭제합니다. Subversion의 소스 트리에 있는 tools/backup/hot-backup.py 스크립트는 Berkeley DB 데이터베이스 환경이 동작하는 동안에도 그 내용을 안전하게 백업하는 방법을 보여줍니다. svnadmin hotcopy 명령을 사용하세요. 일반적으로 말해 정말로 편입증이 있는 사람이나 커밋이 일어날 때마다 저장소 전체를 백업할 것입니다. 그러나 저장소가 다른 장황성을 유지하는 구조를 가져 어느 정도의 사소한 입도를(커밋 마다의 email와 같이) 가지고 있으면 데이터베이스의 온라인 백업은 저장소(repository) 관리자가 매일 저녁 하는 시스템 백업의 일환으로서 하고 싶어질지도 모릅니다. 한층 더 자주 있는 상황에서는 저장소(repository)의 커밋 email만의 아카이브(archive)는 복구의 충분한 바탕으로 되고 적어도 마지막 몇회분의 커밋에 대해서는 그렇겠지요. 그러나 어쨌든 그것은 당신의 데이터인 (뜻)이유로 좋아할 뿐(만큼) 충분한 백업을 취해 주세요. Berkeley DB 는 또 데이터베이스 테이블을 ASCII 텍스트 파일로 변환하거나 그 역의 변환을 하는 유틸리티를 가지고 있습니다. db_dump 와 db_load 프로그램은 Berkeley DB 데이터베이스 의 키와 값을 표현하는 커스텀 형식 파일의 읽고 쓰기를 실행합니다. Berkeley 데이터베이스는 머신 아키텍처를 또 있고다 호환성 (이)가 있으므로 이 형식은 아키텍처나 OS의 차이를 의식하지 않고 데이터베이스 머신간에 전송 하는데 편리한 방법입니다. 저장소 청소 Subversion 저장소는 일반적으로 일단 한 번 설정하면 그다지 주의를 기울일 필요는 없습니다. 그러나 관리자에 의한 몇개의 보조가 필요할지도 모릅니다. svnadmin 유틸리티 에는 다음과 같은 작업을 돕기 위한 기능이 있습니다. 커밋 로그 메시지 수정 혹은 트랜잭션(transaction) 삭제 굳어져 버린 저장소(repository) 복구 저장소의 내용을 다른 저장소로 옮기기 svnadmin의 하위 명령어로 가장 자주 사용되는 것은 아마 setlog일 것입니다. 트랜잭션이 저장소(repository)에 커밋되어 새로운 리비전을 만들었을 때, 로그 메시지는 그 리비전 자체의 버전화 되지 않은 속성으로서 저장됩니다. 바꾸어 말하면 저장소는 그 속성의 마지막 값만을 기억하고 있고 이전 것은 버리게 됩니다. 가끔 사용자는 로그 메시지에서 실수를 찾아냅니다(오자나 잘못된 정보 등). 만약 저장소가(pre-revprop-change 와 post-revprop-change 혹은 사용해. >참조) 커밋 완료 후 이 로그 메시지의 변경을 받아들이도록 설정되었다면 사용자는 svn 프로그램의 propset 명령어를 사용해 로그 메시지를 원적으로 수정 할 수 있습니다. (>참조) 그러나, 정보가 영원히 없어지는 일을 막기 위해 Subversion 저장소는 기본값으로 그 기능이 꺼져있습니다. 버전화 되지 않는 속성은 관리자만 변경할 수 있도록 기본값으로 설정되어 있습니다. 만약 관리자가 로그 메시지를 변경할 필요가 있는 경우 svnadmin setlog를 사용합니다. 이 명령어는 저장소(repository)의 지정한 리비전의 로그 메시지(svn:log 속성)을 지정한 파일로부터 읽어들이 값으로 바꿉니다. \$ echo "Here is the new, correct log message" > newlog.txt \$ svnadmin setlog myrepos newlog.txt -r 388 그 로그 메시지를 버리는군! 관

리자는 svnadmin setlog 는 저장소(repository)에 할 것이었다 어떠한 리비전 자산의 폭스 클립트 도 우회 해 버리는 것에 주의해야 합니다. 이 하위 명령어에는 수정 대상으로 해 올바른 리비전 번호를 지정해 있는 것을 확인하고 등 세심의 주위를 기울여 주세요. 다른 자주 있는 svnadmin 의 사용법은 종료하고 있지 않다 아마 죽어 버린Subversion 트랜잭션(transaction)에 관한 저장소(repository)에의 문의입니다. 커밋이 실패했을 때 보통 트랜잭션(transaction)는 예쁘게 소거됩니다. 즉 트랜잭션(transaction) (은)는 저장소(repository)로부터 삭제되어 그 트랜잭션(transaction)에(인 만큼) 관련했다 데이터도 이와 같이 삭제됩니다. 그러나, 자주 트랜잭션(transaction)의 청소가 일어나지 않고 실패하는 것이 있습니다. 이것에는 몇개의 이유가 생각됩니다: 아마 클라이언트의 조작이 유저에 의해 난폭하게 종료되었는지 네트워크의 이상등이 처리의 도중에 일어났을 경우입니다. 이유에 관계없이 이러한 죽었다 트랜잭션(transaction)는 저장소(repository)를 어질러 디스크를 먹을 뿐입니다. svnadmin의lstxns 커멘드 (을)를 사용해 그 시점에서의 미완료의 트랜잭션(transaction)의 이름의 일람표시 할 수가 있습니다. \$ svnadmin lstxns myrepos 19 3a1 a45 \$ 출력 결과의 각각의 항목은svnlook (와 그--transaction 옵션)(으)로 사용할 수가 있어 누가 트랜잭션(transaction)를 만들어 그것은 언제로 어떠한 변경이 트랜잭션(transaction)에 일어났는지 를 알 수가 있습니다. 바꾸어 말하면(자) 그 트랜잭션(transaction)는 삭제 대상으로 해 안전한 후보인가 부디라고 하는 것을입니다. 만약 그러면 트랜잭션(transaction)의 이름을svnadmin rmtxns 에 건네줄 수가 있어 그 트랜잭션(transaction)는 예쁘게 삭제 됩니다. rmtxns 하위 명령어는 lstxns의 출력을 그대로 입력으로서 취할 수도 있습니다! \$ svnadmin rmtxns myrepos `svnadmin lstxns myrepos` \$ 이러한 두 하위 명령어를 사용하는 경우 저장소(repository)를 일시적으로 클라이언트로부터 액세스 할 수 없게 할 필요가 있습니다. 이것으로 아무도 당신이 클린 업을 시작하기 전에 올바른 트랜잭션(transaction) (을)를 개시할 수 없게 됩니다. 이하는 저장소(repository)내의 미해결의 트랜잭션(transaction) 의 각각 붙은 정보를 재빠르게 생성하기 위한 약간의 스크립트입니다: txn-info.sh (미해결 트랜잭션(transaction)의 표시) #! /bin/sh ### Generate informational output for all outstanding transactions in ### a Subversion repository SVNADMIN=/usr/local/bin/svnadmin SVNLOOK=/usr/local/bin/svnlook REPOS=\${1} if [x\$REPOS = x]; then echo "usage: \$0 REPOS_PATH" exit fi for TXN in ` \${SVNADMIN} lstxns \${REPOS} `; do echo "----[Transaction \${TXN}]-----" "\${SVNLOOK} info \${REPOS} --transaction \${TXN} done 이 스크립트를 /path/to/txn-info.sh /path/to/repos와 같이 해 실행할 수 있습니다. 출력은 기본적으로는svnlook info 출력의 여러가지 단편을 이은 것 같은 것이 됩니다. (>참조) 이하와 같은 느낌입니다: \$ txn-info.sh myrepos ---[Transaction 19]----- sally 2001-09-04 11:57:19 -0500 (Tue, 04 Sep 2001) 0 ---[Transaction 3a1]----- harry 2001-09-10 16:50:30 -0500 (Mon, 10 Sep 2001) 39 Trying to commit over a faulty network. ---[Transaction a45]----- sally 2001-09-12 11:09:28 -0500 (Wed, 12 Sep 2001) 0 \$ 보통은 로그 메시지를 갖고 있지 않은 죽은 트랜잭션(transaction)가 보이는 경우 갱신 (혹은 거기에 답은) 조작에 실패한 결과입니다. 이러한 조작은 작업 카피 상태(hood to mimic)상태하 그리고 Subversion의 트랜잭션(transaction)를 이용합니다. 커밋하는 의도가 전혀 없기 때문에 Subversion 은 그러한 트랜잭션(transaction)에 대한다 로그 메시지를 요구하지 않습니다. 로그 메시지가 도착한 트랜잭션(transaction) (은)는 아마 확실히 어떤 종류의 커밋에 실패했을 경우입니다. 또 트랜잭션(transaction)의 타임 스탬프는 흥미로운 정보가 됩니다 예를 들어 어쨌서 9개월이나 전에 시작한 조작이 아직 액티브하겠지 인가? 그렇다고 하는 상태입니다. 간단하게 말해 트랜잭션(transaction)의 클린 업의 결정은 무분별하게 할 필요는 없습니다. 여러가지 정보원아파치의 에러 로그나 액세스 로그 성공한 Subversion의 커밋 로그 등 등가 어떻게 하면 좋은가를 결정하는데 있어서 도움이 됩니다. 마지막으로 관리자는 자주 죽은 트랜잭션(transaction)의 소유자라고 생각되는 사람과(email등으로) 그 죽음에 걸린 트랜잭션(transaction) 상태를 확인할 수가 있습니다. 저장소(repository)의 복구 저장소(repository)의 데이터를 지키기 위해서 데이터베이스 연구 최종 단계는 락의 구조를 이용합니다. 이 구조는 데이터베이스가 있다 부분은 복수의 데이터베이스에 액세스 하고 있는 사람으로부터 동시에 변경하는 것이 없는 것을 프로텍션해 각각의 프로세스는 데이터베이스 (으)로부터 읽힌 데이터가 항상 올바른 상태에 있는 것처럼 보이는 것을 프로텍션합니다. 프로세스가 데이터베이스중의 무엇인가를 변경할 필요가 있다 경우 그것은 우선 목적의 데이터가 락되어 있지 않은지 어떤지를 확인 합니다. 만약 데이터가 락되어 있지 않으면 그 프로세스는 데이터를 잠그어 필요한 변경을 더해 마지막에 락을 뗍니다. 다른 프로세스는 데이터베이스의 그 부분에 액세스 하는 허가를 얻는 데 전에 락의 해제를 기다리는 것을 강요당합니다. Subversion 저장소(repository)를 사용하는데 있어서 치명적인 에러(디스크가 가득 되거나 메모리가 없어지거나)나 세치기에 의해 데이터베이스에 걸린 락을 삭제할 기회를 없애 버리는 일이 있습니다. 그 결과 연구 최종 단계의 데이터베이스는굳어져버립니다. 이렇게 되었을 때에는 저장소(repository)에의 어떠한 액세스도 영구히 기다리게 된다 것으로 되어 버립니다. (라고 하는 것은 모든 새로운 액세스는 락 하지만 해제되는 것을 기다리지만 그것은 결코 오지 않기 때문 입니다) 우선 그런 것이 저장소(repository)에 일어나도 비명을 지르지 마 주세요. Subversion의 파일 시스템은 데이터베이스 트랜잭션 (와)과 체크 포인트 거기에 사전 저널 기입의 구조를 잘 이용하고 있어 정말로 파멸적인 사건 이외는 예를 들어: 하드 디스크 + 강한 전자장 = 파멸 데이터베이스 환경을 영구히 매장해 떠날 수 없는 것을 프로텍션합니다. 충분히 신경질적인 저장소(repository) 관리자는 무슨응등이나 방법으로 저장소(repository) 데이터의 오프 라인 백업을 취하고 있을지도 모르지 않습니다만 백업 테이프를 restore 해 주고와 시스템 관리 책임자를 부르는 것은 아직입니다. 다음에 이하의 순서를 사용해 저장소(repository)의복구 (을)를 시험해 보세요: 저장소(repository)에 액세스 하고 있는(혹은 하려고 하고 있다) 프로세스가 하나도 없는 것을 확인해 주세요. 네트워크 액세스 가능한 저장소(repository)에서는 이것은 Apache HTTP 서버를 갖다온 한다 일도 의미합니다. 저장소(repository)를 소유해 관리하고 있는 유저가 되어 주세요. svnadmin recover /path/to/repos커멘드를 실행해 주세요. 이하와 같은 출력이 표시된다고 생각합니다: Acquiring exclusive lock on repository db, and running recovery procedures. Please stand by... Recovery completed. The latest repos revision is 19. Subversion 서버의 재기동 이 방법은 대부분의 저장소(repository) 락을 해소합니다. 이 커멘드는 단지 root가 되는 것이 아니라 데이터베이스 (을)를 소유해 관리하고 있는 유저로 실행하는 것에 주의해 주세요. 복구 작업은 상처를 입은 여러가지 데이터베이스 파일 (으)로부터의 제작성의 작업도 포함합니다. (예를 들어 공유 메모리 area 등입니다) root 에서의 복구는 root 가 소유하고 있는 파일을 작성하는 것으로 이것은 저장소(repository)에의 접속 상황이 복구한 다음에도 통상의 유저는 이것에 대해서 액세스 한다 일을 할 수 없는 것을 의미합니다. 만약 지금 말한 작업이 무엇인가의 이유로 잘 저장소(repository)를 정상적으로 되돌릴 수 없는 경우 둘을 해야 합니다. 우선 망가진 저장소(repository)를 치워 마지막 백업을 restore 합니다. 그리고 Subversion 의 개발 리스트에 email 합니다. (이것은 dev@subversion.tigris.org입니다) 이 때 문제점을 자세하게 설명해 주세요. 데이터의 일관성은 Subversion 개발자에게 있어 매우 높은 priority입니다. 저장소(repository)의

이행 Subversion 파일 시스템은 다양한 데이터베이스 테이블에 분산되었다 데이터를 가집니다만 이것은 일반적으로는 Subversion 개발자만이 알고 있다 (라고 흥미가 있다) 일입니다. 그러나 모든 혹은 일부의 데이터를 하나의 운반에 편리한 단순한 파일 형식에 정리하고 싶은 것이 있습니다. Subversion 는 그러한 구조를 `svnadmin` 하위 명령어 의 조에 의해 실장하고 있습니다: `dump` 와 `load`입니다. Subversion 저장소(repository)를 덤프 하거나 로드하거나 하는 제일 자주 있는 이유는 Subversion 자신의 변경에 있습니다. Subversion이 완성에 가까워지는 것에 따라 어떤 종류의 변경이 연구 최종 단계 데이터베이스의 schema로 변경될 때 저장소(repository)의 전의 버전과의 호환성이 없어져 버립니다. 이러한 호환성의 경계를 넘어 업그레이드 할 경우에 권하는 방법은 비교적 단순한 것입니다: 현재 버전의 `svnadmin` (을)를 사용해 저장소(repository)를 덤프 파일에 덤프 해 주세요. Subversion의 새로운 버전에서의 업그레이드, 낡은 저장소(repository)를 치워 새로운 하늘의 저장소(repository)를 거기에 만듭니다만 이것에는 새로운 `svnadmin` (을)를 사용해 주세요. 한번 더 새로운 `svnadmin`를 사용해 덤프 파일을 각각 만든지 얼마 안된 저장소(repository)에 로드해 주세요. 마지막으로 낡은 저장소(repository)로부터 새로운 것에 필요한 커스터마이징 부분을 모두 카피해 주세요. 이것에는 `DB_CONFIG` 파일과 혹은 스크립트가 포함됩니다. 새로운 릴리스의 Subversion의 릴리스 노트에 주의해 마지막 업그레이드로 혹은 (이)나 설정 옵션으로 변경이 있는지 없는지를 봐 주세요. `svnadmin dump` 는 저장소(repository) 리비전이 있다 범위를 출력합니다만 그것은 Subversion의 커스텀 파일 시스템 덤프 형식이 되어 있는 것입니다. 덤프 형식은 표준 출력에 표시되어 진행 상황등의 메시지는 표준 에러 출력에 표시됩니다. 이것으로 출력을 파일에 리디렉트 할 수가 있어 그 한편으로 스테이터스의 출력에 대해서는 단말 윈도우상에서 볼 수가 있습니다. 예를 들어: `$ svnlook youngest myrepos 26 $ svnadmin dump myrepos dumpfile` * Dumped revision 0. * Dumped revision 1. * Dumped revision 2. * Dumped revision 25. * Dumped revision 26. 처리의 최후로 지정한 범위의 저장소(repository) 리비전의 데이터 모두 하지만 보존된 하나의 파일(전의 예에서는 `dumpfile`) (을)를 손에 넣을 수가 있습니다. 조가 된 이제(벌써) 한편의 하위 명령어인 `svnadmin load` 는 표준 입력을 Subversion 저장소(repository)의 덤프 파일과 해 해석해 덤프 된 리비전을 목적의 저장소(repository)에 재현합니다. 그것은 또 경과 정보등을 돌려줍니다만 이쪽은 표준 출력에 표시합니다: `$ svnadmin load newrepos dumpfile` Started new txn, based on original revision 1 * adding path : A ... done. * adding path : A/B ... done. ----- Committed new rev 1 (loaded from original rev 1) Started new txn, based on original revision 2 * editing path : A/mu ... done. * editing path : A/D/G/rho ... done. ----- Committed new rev 2 (loaded from original rev 2) Started new txn, based on original revision 25 * editing path : A/D/gamma ... done. ----- Committed new rev 25 (loaded from original rev 25) Started new txn, based on original revision 26 * adding path : A/Z/zeta ... done. * editing path : A/mu ... done. ----- Committed new rev 26 (loaded from original rev 26)

`svnadmin` 는 표준 입력과 표준 출력을 저장소(repository)의 덤프 (와)파 로드 처리에 사용하므로 몇이 있는 사람은 이하와 같은 방식을 시험한다 일도 할 수 있습니다(아마 파이프의 양측의 `svnadmin` (은)는 다른 버전일지도 모릅니다): `$ svnadmin create newrepos $ svnadmin dump myrepos | svnadmin load newrepos` 전에 주의한 것처럼 `svnadmin dump` 는 리비전의 범위를 출력합니다. --revision 옵션을 사용하면 하나의 리비전의 덤프나 리비전 범위의 덤프가 생깁니다. 이 옵션을 생략 하면 모든 존재하는 저장소(repository) 리비전이 덤프 됩니다. `$ svnadmin dump myrepos --revision 23 > rev-23.dumpfile` `$ svnadmin dump myrepos --revision 100:200 > revs-100-200.dumpfile` Subversion은 각각의 새로운 리비전을 덤프 하므로 그 출력에는 다음에 실행되는 로더가 전의 리비전을 바탕으로 해 그 리비전을 재 작성하는데 필요한 충분한 정보가 있습니다. 바꾸어 말하면(자) 덤프 파일중에서 어떠한 리비전이 지정되어도 리비전중에서 변경이 있던 아이템만이 덤프에 나타난다고 하는 것 입니다. 이 규칙의 유일한 예외는 현재의 `svnadmin dump` 하지만 덤프 하는 최초의 리비전입니다. 디폴트에서는 Subversion은 전의 리비전에 대한 단순한 차분으로서 최초의 덤프 리비전을 표현할 것은 없습니다. 이 이유의 하나는 덤프 파일에는 직전의 리비전이 없기 때문입니다! 두번째에 Subversion (은)는 덤프 데이터가 로드 되는 저장소(repository) 상태에 대해 아무것도 모른다 (으)로부터입니다. (만약 로드가 일어난다고 하면 입니다만.) `svnadmin dump` 의 개별의 출력의 출력이 자기 충족 해 있는 것을 프로텍션하기 위해(때문에) 최초의 덤프 리비전은 디폴트에서는 모든 디렉토리 파일, 저장소(repository)에 있는 그 리비전의 속성 의 완전한 표현이 되어 있습니다. 그러나 이 디폴트의 행동을 바꿀 수도 있습니다. 저장소(repository)를 덤프 할 경우에 --incremental 옵션을 추가하면(자) `svnadmin` 는 최초의 덤프 리비전과 저장소(repository)중의 직전 리비전과의 차분을 이라고 깔때기 합니다. 나머지의 모든 덤프 되는 리비전에도 같은 방법으로 취급합니다. 그리고 덤프 범위에 있는 나머지의 리비전이 출력하는 것과 같이 최초의 리비전부터 리비전중에서 일어나는 변경만을 고려해 출력합니다. 이 이전은 큰 하나의 덤프 파일의 대신에 로드되는 것 같은 작은 얼마든지의 덤프 파일을 만들 수가 있는 것입니다. 이런 느낌입니다 : `$ svnadmin dump myrepos --revision 0:1000 > dumpfile1 $ svnadmin dump myrepos --revision 1001:2000 --incremental > dumpfile2 $ svnadmin dump myrepos --revision 2001:3000 --incremental > dumpfile3` 이러한 덤프 파일은 이하와 같은 커멘트가 흘러 나오고 새로운 저장소(repository)중에 로드 됩니다: `$ svnadmin load newrepos < dumpfile1 $ svnadmin load newrepos < dumpfile2 $ svnadmin load newrepos < dumpfile3 --incremental` 옵션을 사용한 다른 근사한 방법은 벌써 존재하고 있는 덤프 파일에 새로운 덤프 리비전 범위를 추가하는 것입니다. 예를 들어 `post-commit` 훅이 있어 그것은 단지 훅을 트리거하는 것 같은 하나의 리비전의 저장소(repository) 덤프를 추가하는 것입니다. 혹은 이하와 같은 스크립트 (이)가 있어 매일 저녁 마지막에 이 스크립트를 실행하고 나서 리포지토리 에 추가된 모든 리비전의 덤프 파일 데이터를 추가하는 것입니다. 저장소(repository)의 차분 덤프의 이용 `#!/usr/bin/perl $repos_path = '/path/to/repos'; $dumpfile = '/usr/backup/svn-dumpfile'; $last_dumped = '/var/log/svn-last-dumped'; # Figure out the starting revision (0 if we cannot read the last-dumped file, # else use the revision in that file incremented by 1). if (open LASTDUMPED, "$last_dumped") { $new_start = LASTDUMPED; chomp $new_start; $new_start++; } else { $new_start = 0; } # Query the youngest revision in the repos. $youngest = `svnlook youngest $repos_path`; chomp $youngest; # Do the backup. `svnadmin dump $repos_path --revision $new_start:$youngest --incremental >> $dumpfile`; # Store a new last-dumped revision open LASTDUMPED, " $last_dumped" or die; print LASTDUMPED "$youngest\n"; close LASTDUMPED; # All done! 이와 같이 이용하는 것으로 svnadmin의 dump 와 load 커멘드는 가치가 있는 수단이 됩니다만 이것에 의해 저장소(repository)의 변경을 시간을 들여 백업 해 시스템 크래쉬나 다른 괴멸적인 사건에 갇춘다는 것입니다. 마지막으로 Subversion의 저장소(repository) 덤프 파일 형식의 다른 이용 방법과 해서는 다른 보존의 구조나 버전 컨트롤 시스템으로부터의 변환입니다. 덤프 파일 형식은 대부분이 인간이 읽을 수 있는 형태가 되어 있으므로 Subversion의 저장소(repository) 덤프 파일 형식은 RFC-822 형식에 자주(잘) 비슷해 대부분의 email 로 이용되고 있는 것과 같은 형식입니다. 이 파일 형식을 사용하면(자) 비교적 간단하게 일반적인 변경접세트`

를 표현할 수가 있는 각각의 변경은 새로운 리비전으로서 다루어집니다. 저장소(repository)의 백업 현대적인 컴퓨터가 태어나고 나서 기술적으로는 매우 발전해 왔지만 유감스럽게 하나의 일만은 틀림없이 진실합니다. 다가끔 모든 것은 완전히 엉망이 되어 버린다 라고 하는 것입니다. 정전 네트워크 절단 RAM의 파괴 하드 디스크의 크래쉬는 마귀 이외의 누구이기도 하지 않습니다. 운명은 가장 뛰어난 관리자에게조차 닥칩니다. 그래서 매우 중요한 토끼에 도착합니다. 어떻게 저장소(repository)의 백업을 취하는지입니다. 일반적으로 Subversion의 저장소(repository) 관리자에게 있어 두 백업 방법이 있습니다. 차등 백업과 풀백 업입니다. 이 장의 전의 마디로 어떻게 `svnadmin dump --incremental` 를 사용해 차분 백업을 취하는지를 논의했습니다 (>참조). 본질적으로 이 아이디어는 마지막에 백업을 취하고 나서 둔 저장소(repository)의 변경 부분만큼을 백업 하는 방법입니다. 저장소(repository)의 풀백 업은 문자 그대로 저장소(repository) 디렉토리 전체의 복제를 만드는 것입니다(이것은 Berkeley 데이터베이스 환경도 포함됩니다) 그런데 일시적으로 저장소(repository)에 대한 모든 액세스를 금지하지 않으면 단순한 재귀적인 디렉토리 카피의 실행은 죽은 백업을 만들어 끝내는 위험을 갖고 있습니다. 그렇다고 하는 것은 누군가가 병행해 데이터베이스에 기입해 있을지도 모르기 때문입니다. 행운의 일로 Sleepycat의 BerkeleyDB 문서에서는 있는 정해진 카피의 순서가 쓰여져 있습니다. 그 순서에 따르면 데이터베이스 파일은 올바른 백업 카피가 되는 것을 프로텍션하는 형태로 카피할 수 있습니다. 한층 더 능숙한 것에 그 알고리즘은 당신이 실장할 필요는 없고 벌써 Subversion 개발 팀이 하고 있다고 하는 것입니다. `hot-backup.py` 스크립트는 Subversion의 소스 패키지의 `tools/backup/` 디렉토리에 있습니다. 저장소(repository) 패스와 백업 위치를 지정하면(자) `hot-backup.py` (은)는 동작중의 저장소(repository)를 핵 올라가는데 필요한 스텝을 실행합니다. 당신에게 저장소(repository) 액세스를 금지하는 것 없이입니다. 그 다음에 동작중의 저장소(repository)로부터 죽어 있는 Berkeley 로그 파일을 예쁘게 삭제합니다. 차등 백업이 있다고 해도 규칙적으로 이 프로그램을 실행하고 싶고 될지도 모릅니다. 예를 들어 `hot-backup.py` 를 프로그램 스케줄러에 추가하려고 생각할지도 모릅니다(Unix 이면 `cron` 와 같은 것). 혹은 세세한 입도의 백업을 좋아하면 `hot-backup.py` (을)를 부르는 것 같은 `post-commit` 폭스 클립트를 쓸 수도 있습니다. (>참조). 이것은 새로운 리비전이 작 히에 저장소(repository)의 새로운 백업이 생기는 방식입니다. 단지 이하를 동작중의 저장소(repository) 디렉토리에 있다 `hookspost-commit` 에 추가해 주세요: `(cd /path/to/hook/scripts; ./hot-backup.py ${REPOS}/path/to/backups )` 결과의 백업은 완전하게 기능하는 Subversion 저장소(repository)로 현행의 저장소(repository)이 무엇인가 심하게 되었을 때에는 옮겨놓아 사용하는 것이 할 수 있는 것입니다. 양쪽 모두의 백업 방법에는 각각 이점이 있습니다. 제일 간단한 것은 풀백 업으로 그것은 항상 현행 저장소(repository)의 완전한 카피입니다. 반복이 됩니다만 무엇인가 안되는 일이 동작중의 저장소(repository)에 일어났다면 또는 단순한 재귀적인 디렉토리 카피로 이 백업을 복원할 수가 있습니다. 유감스럽게 만약 저장소(repository)의 복수의 백업(을)을 관리하고 있는 경우 이러한 풀 카피는 실행중의 저장소(repository)와 같은 정도 각각이 디스크를 먹는다고 하는 것입니다. 저장소(repository) 덤프 형식을 사용한 차등 백업은 데이터베이스 schema 하지만 계속하는 Subversion 자신의 버전간에 변경될 때에는 매우 도움이 됩니다. 저장소(repository)의 풀 덤프와 로드는 일반적으로 저장소(repository)를 새로운 schema에 업그레이드 하는 것이 필요합니다. 그러한 작업의 반(즉 덤프의 부분)에 대해서는 벌써 끝나고 있으므로 매우 편리합니다. 불행하게도 차등 백업의 작성그리고 그 restore는 긴 시간이 걸립니다만 그것은 각각의 커밋이 덤프 파일 또는 저장소(repository)의 안으로 실제로 재실행되기 때문입니다. 어느 쪽의 백업의 경우도 저장소(repository) 관리자는 어떻게 해 버전화 되지 않는 속성에의 변경이 백업에 영향을 줄까에 주의할 필요가 있습니다. 이러한 변경은 새로운 리비전을 그 자체로 만들어 내는 것은 아니기 때문에 `post-commit` 훅을 호출하는 계기 에는 안되어 `pre-revprop-change` 이나 `post-revprop-change` 훅의 계기 예조차 되지 않을 것입니다. 예를 들어 `svnadmin setlog`는 어쨌든 훅 인터페이스를 우회 하므로 했다. 그리고 시간의 순서에 따르지 말고 리비전 속성을 변경할 수가 있다 언제라도 어느 리비전 속성을 변경할 수가 있는 의로 마지막 몇개의 리비전의 차등 백업은 그 이전의 백업의 일부로서 행해진 리비전 속성의 수정은 거두어 들인다 일을 할 수 없습니다. 자주 저장소(repository)의 백업에 대한 최선의 방법은 분산 시키는 것입니다. 풀백 업과 차등 백업에 커밋 email의 아카이브(archive)를 추가할 수가 있습니다. 예를 들어 Subversion 개발자의 Subversion 원시 코드 저장소(repository)를 새로운 리비전 하지만 만들어질 때마다 백업 합니다. 그리고 모든 커밋과 속성 변경의 통지 email를 아카이브(archive) 해 취해 둡니다. 같은 방법을 취해 주세요. 다만, 필요한 범위에서 편리함과 안전성의 미묘한 장미 스텝을 취해 주세요. 그리고 이러한 일을 전부 해도 운명의 철권으로부터 하드웨어를 지킬 수 없는 것에 주의해 주세요. 아시는 바입니다 그녀의 모든와 우연을 나타내는 집합명사입니다. 백업은 확실히 그러한 시련때부터 당신을 구할 것입니다. 저장소(repository)의 네트워크 Subversion 저장소(repository)는 저장소(repository)가 있는 머신상에서 실행되고 있는 클라이언트(으)로부터 액세스 할 수가 있습니다. 그러나, 전형적인 Subversion의 설정은 어느 하나의 서버머신에 대해서 오피스 전체에 있는 컴퓨터상의 클라이언트로부터 액세스 된다고 하는 것입니다. 혹은 전세계로부터. 이 마디에서는 밖의 세계에 열려 있는 Subversion 저장소(repository)를 리모트 클라이언트로부터 이용되는 호스트 머신에 어떻게 해 짓는가 한다 일에 대해 설명합니다. 현실점에서의 Subversion로 이용할 수 있는 서버의 구조를 설명해 설정과 그 사용법을 논의합니다. 이 마디를 읽은 다음에는 자신의 요구를 채우기 위해서(때문에)는 어떻게 네트워크를 설정하면 좋은가(을)를 판단할 수가 있어 당신의 호스트 머신상에서는 어떻게 그 설정을 하든 좋은가를 이해할 수 있다고 생각합니다. `httpd` Apache HTTP 서버 Subversion의 주된 네트워크 서버는 WebDAV/deltaV 프로토콜로 통신한다 일을 할 수 있는 Apache HTTP 서버입니다(`httpd`). 이 프로토콜은(HTTP 1.1의 확장입니다만 <http://www.webdav.org/>(참조) World Wide Web의 핵심이 되는 흔히 있던 HTTP 프로토콜을 필요로 해 게다가 글기능특수한 버전화 된 글기능을 추가 한 것입니다. 그 결과는 표준화 된 견뢰한 시스템이며 Apache2.0의 일부로서 패키지화되고 있습니다. 이것은 많은 유명한 오피레이팅(operating) 시스템이나 서드 파티 제품의 일부로서 서포트되고 있습니다. 되에 그것은 특별한 관리자를 필요로 하지 않습니다. 그렇게 하는 것 그들은 정말로 싫어하고 있습니다. 이하의 논의는 Apache 설정의 레퍼런스도 포함하고 있습니다. 몇개의 예는 그러한 인스트럭션의 사용법입니다만 완전한 사용법은 이 장의 범위를 넘고 있습니다. Apache 팀은 매우 우수한 문서를 관리하고 있어 그들의 웹 사이트 그리고 공개하고 있습니다. 그것은 <http://httpd.apache.org>입니다. 예를 들어 설정에 관한 일반적인 레퍼런스는 <http://httpd.apache.org/docs-2.0/mod/directives.html>. 게다가 Apache의 설정을 변경하는 경우 도중에 잘못된는 일도 있습니다. Apache의 로그 하부조직에 자세히 않다면 거기에 조심해 주세요. `httpd.conf`파일에는 Apache가 생성하는 액세스 로그와 에러 로그의 디스크상의 장소를 지정하는 인스트럭션이 있습니다. (각각 `CustomLog` 와 `ErrorLog` 인스트럭션이 됩니다.) Subversion의 `mod_dav_svn`는 Apache의 에라로그인타페스도 사용합니다. 간단하게는 모르는 것 같은 문제를 분명히 하려면 정보원(으)로서 그러한 파일의 내용을 언제라도 볼 수가 있습니다. HTTP 베이스의 저장소(repository) 액세스로 필요한 일 저장소(repository)를 HTTP 넘어로 네트워크화하는 경우 기본적으로는 두 패키지에 있는 네 개의 부품이 필요하게 됩니다. Apache `httpd 2.0` 그것과 함께 배포되는 `mod_dav` DAV 모듈, Subversion 본체, 그리고, Subversion과 함께 배포되는 `mod_dav_svn`파일 시스템 제공 모듈

입니다. 이러한 부품을 모두 손에 넣으면 저장소(repository)의 네트워크화는 이하와 같이 간단하게 할 수 있습니다: httpd 2.0 이후를 손에 넣어 mod_dav를 유효하게 해 실행합니다. mod_dav_svn 를 mod_dav 플러그인으로서 인스톨 합니다. 이것은 Subversion의 프로그램 라이브러리를 사용해 저장소(repository)에 액세스 합니다. 그리고 httpd.conf파일을 저장소(repository)를 공개하도록(듯이) 설정합니다. 최초의 두 개의 작업은 httpd 와 Subversion을 원시 코드로부터 컴파일 하는지 벌써 시스템에 인스톨 되고 있는 바이너리 패키지를 사용하면 할 수 있습니다. Subversion을 Apache HTTP 서버와 함께 이용하기 위해서 어떻게 컴파일 할까에 붙은 최신의 정보와 이 목적을 위해서(때문에) Apache 자신을 어떻게 설정하면 좋은 것처럼 붙어 Subversion 원시 코드 트리의INSTALL 파일을 봐 주세요. 기본적인 Apache의 설정 시스템에 필요한 부품을 모두 인스톨 하면 나머지는 Apache 의 설정을httpd.conf 로 할 뿐입니다. mod_dav_svn 모듈을 로드하도록(듯이) Apache에 지시하려면 LoadModule 인스트럭션을 사용합니다. 이 인스트럭션은 다른 Subversion 관련 설정에 앞서 지정할 필요가 있습니다. 만약 Apache가 디폴트 설정으로 인스톨 되고 있다 경우 mod_dav_svn 모듈은 Apache 인스톨 디렉토리(대부분 /usr/local/apache2 (와)과 같은 장소입니다)의 modules서브 디렉토리에 인스톨 되고 있을 것입니다. LoadModule 인스트럭션은 단순한 구문을 갖고 이름이 붙은 모듈을 디스크중의 공유 프로그램 라이브러리의 장소에 연결시킵니다: LoadModule dav_svn_module modules/mod_dav_svn.so 만약mod_dav 가 공유 오브젝트로서 컴파일 되고 있다면 (httpd 바이너리 에 직접 정적으로 링크 되고 있는 것이 아니라) 같은 LoadModule 가 그 때문에(위해) 역시 필요합니다. 이것으로 설정 파일의 뒤 쪽에 Apache 에 대해 Subversion의 저장소(repository)가 어디에 있는지를 지정할 필요가 있습니다. Location 인스트럭션은 XML풍의 구문을 갖고 있어 개시 태그로 시작되어 종료 태그로 끝납니다. 이전에 다양한 설정 인스트럭션을 봅니다. Location인스트럭션의 목적은 Apache 가 그 중의 어느인가의 URL에 대하는 것인 경우에는 특별한 처리를 하도록(듯이) 지정하는 것입니다. Subversion의 경우 Apache에 DAV 층의 비전화 된 리소스에 대해서는 URL를 단지 건네주도록(듯이) 할 필요가 있습니다. 아파치에 /repos/ 그리고 시작되는 패스를 가지는 모든 URL에 대해서 /absolute/path/to/repository 의 장소에 저장소(repository)를 가진 DAV 프로바이더에 제어를 건네주도록(듯이) 설정한다 일이 생깁니다. 거기에는 다음과 같은 httpd.conf 구문을 사용합니다: Location /repos DAV svn SVNPath /absolute/path/to/repository /Location 만약 복수의 Subversion 저장소(repository)를 서포트하려고 하고 있어 그것이 로컬 디스크상의 같은 친디렉토리를 가지는 경우 다른 인스트럭션을 사용할 수가 있습니다. 그것은SVNParentPath 인스트럭션으로 공통의 친디렉토리를 지정합니다. 예를 들어 만약 복수의 Subversion 저장소(repository)를usr/local/svn 디렉토리에 만들어http://my.server.com/svn/repos1라든지 http://my.server.com/svn/repos2라든지 말하는 것 같은 URL로 액세스 하고 싶은 경우 이하의 예의 같은 httpd.conf설정 구문을 사용할 수가 있습니다: Location /svn DAV svn SVNParentPath /usr/local/svn /Location 이 구문을 사용하면 Apache 는 패스 부분이/svn/ 로 시작되는 모든 URL를 Subversion의 DAV 프로바이더에 건넵니다. 이 때 SVNParentPath 인스트럭션으로 지정된 디렉토리 안의 아이탬은 실제의 Subversion 저장소(repository)이라고 해석됩니다. 이것은 매우 편리한 구문입니다. 그것은 SVNPath 인스트럭션과는 달라 새로운 저장소(repository)를 만드는데 Apache를 재기동한다 필요가 없기 때문입니다. 퍼미션, 인증, 허가 여기서 퍼미션에 대한 강한 의문이 일어납니다. Apache를 실행한 경험이 있다면 벌써 당신은 여러가지 컨텐츠를 가지고 있다고 생각합니다웹페이지, 스크립트, 그 외 여러가지 입니다. 이러한 아이탬은 벌써 Apache 부하로 동작하기 위한 혹은 좀 더 정확하게는 아파치가 그러한 파일과 함께 동작하기 위해(때문에) 의 퍼미션이 설정되어 있습니다. Apache가 Subversion 서버로서 이용되는 경우 Subversion 저장소(repository)에 대한 읽고 쓰기의 퍼미션도 설정할 필요가 있습니다. 벌써 있는 다른 웹페이지나 스크립트의 설정을 부수지 않고 Subversion 저장소(repository)에 대한 퍼미션의 시스템 설정을 할 필요가 있습니다. 이것은 Subversion 저장소(repository)의 퍼미션의 변경은 Apache가 제공 하고 있는 다른 것과 잘 성냥 하도록(듯이) 하지 않으면 안 되는 혹은 httpd.conf 중의 User 와 Group인스트럭션으로 Apache가 Subversion 저장소(repository)의 소유자인 것 같은 유저/그룹에서 실행해야 한다고 말하는 것입니다. 퍼미션을 설정하는 유일한 올바른 사용 방법이라고 한 것은 않고 관리자는 어느 특성의 방식을 취한다 각각의 이유가 있을 것입니다. 말하고 싶은 것은 퍼미션에 관련한 문제는 아마 Subversion 저장소(repository)를 Apache로 사용하는데 있어서 1번 잘 간과해 버리는 부분이라고 하는 것입니다. 그리고 퍼미션에 대해 말하는 한편으로 어떻게 하면(자) Apache가 제공하는 인증과 허가 구조를 이번 문제에 잘 적용시키는 것이 할 수 있는가 하는 일도 채택할 필요가 있습니다. 이것에 대해 무엇인가 시스템 전체의 설정을 하지 않으면 Location 인스트럭션을 통해서 이용할 수가 있다 Subversion 저장소(repository)는 누구로부터도 액세스 할 수가 있습니다. 바꾸어 말하면 누구라도 Subversion 클라이언트를 사용해 저장소(repository) URL(와 그 서브 디렉토리)의 작업 카피를 체크 아웃 할 수가 있어 브라우저로 저장소(repository)의 URL를 입력하는 것으로 누구라도 저장소(repository)의 최신 버전을 대화적으로 열람할 수가 있어 누구라도 저장소(repository)에 커밋할 수가 있습니다. 만약 전반적으로 읽어 들여 혹은 기입해 액세스의 제한을 저장소(repository)에 걸치고 싶은 경우 Apache의 편집 액세스 제어 기능을 사용할 수가 있습니다. 그러한 것으로 1번 간단한 기능은 베이직 인증 기구로 그 사람이 그 사람 자신인 것을 확인하기 위해서 유저명으로 패스워드를 사용합니다. Apache에는 htpasswd 유틸리티가 있어 액세스 할 수 있는 유저와 패스워드의 리스트를 관리할 수가 있습니다. Subversion 저장소(repository)에 특별한 액세스 설정을 하고 싶을 때에도 이것을 사용할 수 있습니다. Sally와 Harry에 커밋 권한을 주어 봅시다. 우선 패스워드 파일 에 두 명을 추가하지 않으면 안됩니다. \$ ## First time: use -c to create the file \$ htpasswd -c /etc/svn-auth-file harry New password: ***** Re-type new password: ***** Adding password for user harry \$ htpasswd /etc/svn-auth-file sally New password: ***** Re-type new password: ***** Adding password for user sally \$ 다음에 Location 블록의 안쪽에 httpd.conf 인스트럭션을 몇개인가 추가해 Apache에 이 새로운 패스워드를 사용해 줄 것을 전할 필요가 있습니다. AuthType 인스트럭션은 어떠한 타입의 허가 방법을 사용하는지를 지정합니다. 이번은 Basic 타입을 지정 하고 싶다고 합니다. AuthName 는 허가 도메인을 위해서(때문에) 임의에게 주는 이름입니다. 대부분의 브라우저는 유저에게 이름과 패스워드를 문의할 때 이 이름을 pop-up 다이얼로그 박스중에 표시합니다. 마지막에AuthUserFile 인스트럭션을 사용해 htpasswd로 만든 패스워드 파일의 장소를 지정합니다. 이 세 개의 인스트럭션을 추가한 뒤 Location 블록은 무엇인가 이하와 같이 되어 있을 것입니다: Location /svn DAV svn SVNParentPath /usr/local/svn AuthType Basic AuthName "Subversion repository" AuthUserFile /path/to/users/file /Location 이 시점에서 아파치를 재기동하면 허가가 필요등의 Subversion 조작도 Subversion 클라이언트로부터 유저명으로 패스워드를 수중에 넣는다는 일이 생깁니다. 그것은 이전에 수중에 넣은 값을 사용해도 괜찮고 유저 에 문의하는 형태에서도 괜찮습니다. 그리고는 Apache에 어느 조작에 대해 실제로 허가의 구조를 이용하는지를 지정해 줄 뿐입니다. Location 블록에 Require valid-user 인스트럭션을 추가해 모든 저장소(repository) 조작에 제한을 걸 수가 있습니다. 전의 예에서는 harry 나sally가 올바른 패스워드를 입력했을 때 만여라 Subversion 저장소(repository)의 모든 조작을 허가한다고 하는 것이 됩니다. 가끔 거기까지 어려운 제한은 필요없는 것

이 있습니다. 예를 들어 `http://svn.collab.net/repos/svn` 에 있는 저장소(repository)에는 Subversion의 원시 코드가 있습니다만 온 세상의 누구라도 읽을 보고의 저장소(repository) 조작을 허가하고 있습니다 (작업 카피의 체크아웃이나 브라우저로 저장소(repository)를 열람하거나 등입니다)가 기입 권한은 허가된 유저 밖에 주고 있지 않습니다. 이 손의 선택적인 제한을 걸치려면 `Limit` 와 `LimitExcept` 설정 인스트럭션을 사용할 수가 있습니다. `Location` 인스트럭션과 같이 이러한 블록은 개시와 종료 태그를 갖고 `Location` 블록의 안쪽에 네스트 할 수가 있습니다. `Limit` 와 `LimitExcept` 인스트럭션으로 지정하는 파라미터는 그 블록인 만큼 영향을 주는 것 같은 타입의 것입니다. 예를 들어 현재 서포트하고 있는 읽기 조작 이외의 저장소(repository) 액세스를 금지하고 싶은 경우 `LimitExcept` 인스트럭션을 사용할 수가 있어 이 경우 `GET,PROPFIND,OPTIONS`, 그리고 `REPORT` 를 파라미터로서 건네줍니다. 그리고 전에 주의한 `Require valid-user` 인스트럭션을 단지 `Location` 블록의 안쪽에 두는 대신에 `LimitExcept` 블록의 안쪽에 있어 줍니다. `Location /svn DAV svn SVNParentPath /usr/local/svn AuthType Basic AuthName "Subversion repository" AuthUserFile /path/to/users/file LimitExcept GET PROPFIND OPTIONS REPORT Require valid-user /LimitExcept /Location` 이러한 것은 단순한 예에 지나지 않습니다. Apache 액세스 제어에 대한 좀 더 자세한 정보는 Apache 문서 튜토리얼집 `http://httpd.apache.org/docs-2.0/misc/tutorials.html` 의 `Security` 의 장을 봐 주세요. 서버 명칭과 `COPY` 요구 Subversion은 서버상의 파일이나 디렉토리의 카피를 실행한다 요구 타입으로서 `COPY`를 사용합니다. Apache 모듈의 정합성 체크의 일관으로서 카피원래는 카피처와 같은 머신상에 어느 요구됩니다. 이 요구를 채우기 위해서(때문에) `mod_dav` 에 서버의 호스트 명칭으로서 이용하는 이름을 가르칠 필요가 있습니다. 이것에는 보통 `httpd.conf` 중(안)에서 `ServerName` 인스트럭션을 사용해 줍니다. `ServerName svn.red-bean.com NameVirtualHost` 를 사용한 Apache의 가상 호스트 서포트를 사용하고 있는 경우 `ServerAlias` 인스트럭션 그리고 서버의 추가 명칭을 지정할 필요가 있습니다. 이것에 대해서도 자세하게는 Apache의 문서를 참조해 주세요. 저장소(repository) HEAD의 열람 Subversion 저장소(repository)의 Apache/WebDAV 설정의 이점의 하나에 버전 관리하에 있는 파일이나 디렉토리의 최신 리비전 하지만 보통 웹 브라우저로부터 직접 참조 가능하다고 하는 것입니다. Subversion은 버전화 된 리소스의 특징에 URL를 사용하므로 HTTP 베이스의 저장소(repository) 액세스에 사용되는 URL는 웹 브라우저로 직접 입력 할 수가 있습니다. 브라우저는 URL의 `GET` 요구를 보내 그 URL가 어느 버전 디렉토리나 파일일가에 의거해 `mod_dav_svn`는 디렉토리 일람이나 파일 내용을 돌려줍니다. URL는 보고 싶다고 생각하는 리소스의 버전에 대한 정보는 굳이 포함해 없기 때문에 `mod_dav_svn`는 항상 최신의 버전을 표시합니다. 이것은 기능적으로 봐 훌륭한 부차적인 효과가 있어 그것은 동료에게 문서의 참조로서 지정할 수가 있는 것으로 그 URL는 항상 문서의 최신을 가리키고 있습니다. 물론 다른 웹 사이트로부터의 하이퍼 링크 (으)로서 그 URL를 사용할 수도 있습니다. 버전 관리하에 있는 파일의 URL로부터도 와 여러가지 정보를 얻을 수 있습니다. 결국 거기가 흥미로운 정보가 있을 것 같은 장소입니다. 그렇지만, Subversion의 디렉토리 일람을 참조할 기회가 있을지도 모릅니다. 거기서의 표시에 사용되는 자동 생성된 HTML (은)는 매우 기본적으로 미의식을 만족시키는 것 같은 것은 아니라는 것을 알 수 있으면(자) 생각합니다. 이러한 디렉토리 표시를 커스터마이징 하기 위해서 Subversion (은)는 XML 인텍스 기능을 준비해 있습니다. `httpd.conf` 의 저장소(repository)용 `Location` 블록에 있다 하나의 `SVNIndexXSLT` 인스트럭션은 `mod_dav_svn`에 디렉토리 리스트를 표시할 경우에 XML 출력을 생성해 그 때 지정했다 XSLT 스타일 시트를 참조하도록(듯이) 설정합니다: `Location /svn DAV svn SVNParentPath /usr/local/svn SVNIndexXSLT "/svnindex.xslt" /Location SVNIndexXSLT` 인스트럭션과 잘 할 수 있던 XSLT 스타일 시트를 사용해 디렉토리 표시에 웹 사이트내의 다른 장소에 있는 칼라 스킴이나 이미지를 사용할 수가 있습니다. 혹은 만약 바라다면 Subversion의 소스 패키지의 `tools/xslt/` 에 준비된 샘플 스타일 시트를 사용할 수도 있습니다. `SVNIndexXSLT` 로 지정하는 패스는 실제로는 URL 패스인 것에 주의합니다. 그 스타일 시트를 사용하기 위해서 브라우저가 그것을 읽어들이 수 없으면 안됩니다! 다양한 Apache의 기능 건넨 Web 서버로서의 역할을 완수할 수 있도록 Apache에는 미리 준비된 다양한 기능은 Subversion에 대해도 기능이나 시큐리티를 늘리기 위해서(때문에) 사용할 수가 있습니다. Subversion은 Apache와의 통신에 Neon을 사용합니다. 이것은 SSL(Secure socket layer)와 같은 구조를 서포트하기 위한 일반적인 HTTP/WebDAV 프로그램 라이브러리입니다. (같은 알고리즘이 `gzip` 나 `PKZIP` 로 파일을 좀 더 작은 데이터의 덩어리에 압축하는데 사용되고 있습니다) Subversion나 Apache로 바라는 기능을 사용하는데는 컴파일 하고 나서 그 프로그램 (을)를 적절히 설정할 뿐입니다. 이것은 SSL가 유효한 Subversion 클라이언트는 SSL가 유효하게 되었다 Apache 서버에 액세스 해 모든 통신에 암호화된 프로토콜 (을)를 사용할 수 있다고 하는 것으로 거기에는 URL의 `https:` 의 것인지 비교적 `https:` 를 사용할 뿐입니다. 기업의 파이어 월(fire wall)의 외부로부터 액세스 되는 것이 있는 저장소(repository) (을)를 공개할 필요가 있는 비지니스는 허가되어 있지 않은 외부의 인간이 네트워크 트래픽을 도청 할 가능성을 의식하고 있지 않으면 안됩니다. SSL는 이러한 바람직하지 않다 액세스에 중요한 데이터가 새 가능성을 작게 합니다. Apache는 SSL가 유효가 되고 있는 Subversion 클라이언트만이 저장소(repository)에 액세스 할 수 있도록(듯이) 설정할 수도 있습니다. 압축의 해동은 클라이언트와 서버에 다소의 부하를 걸칩니다만 그것은 실제의 데이터 전송량의 사이즈를 최소로 하기 위해서 압축과 해동이 실행되기 때문입니다. 네트워크의 밴드 폭이 좁을 때로는 이런 종류의 압축은 서버와 클라이언트의 사이에 송신하는 통신의 스피드를 매우 빨리 합니다. 극단적인 경우 이 최소화된 네트워크 전송은 통상이라면 타임 아웃 해 버린다 같은 전송이 완료하는 일도 있습니다. 그만큼 흥미는 끌지 않습니다만 같은 정도 도움이 되는 것은 Apache와 Subversion의 관련에 대한 기능으로 예를 들어 커스텀 포토의 지정(디폴트에서는 80입니다)이나 Subversion 저장소(repository)이 액세스 되는 머신의 버추얼 도메인명칭이나 proxy 경유로 저장소(repository)에 액세스 하는 능력 등입니다. 이것들은 모두 Neon에 의해 준비되어 있으므로 Subversion은 자유롭게 이용할 수가 있습니다. `svnserve`, 커스텀 Subversion 서버 Apache의 대체로서 Subversion은 스탠드얼론의 서버 프로그램 `svnserve` 도 준비해 있습니다. 이 프로그램은 Apache보다 훨씬 가볍게 만들어지고 있어 설정도 훨씬 간단합니다. 보통 TCP/IP접속을 통해서 Subversion 클라이언트와 고유의 프로토콜 그리고 이야기합니다. `svnserve`가 이용한다 일을 할 수 있는 두 개의 기본적인 방식이 있습니다: 인증 없음의(의명의) 액세스 이 시나리오에서는 `svnserve` 프로세스 하지만 서버상에서 실행되어 접속 요구를 기다립니다. `svn` 클라이언트는 고유의 `svn:// URL` 형식을 이용해 접속합니다. 클라이언트 접속은 무조건 받아들여져 저장소(repository)는 인증된 유저 명칭등을 이용하는 일 없이 액세스 됩니다. 자주 관리자는 리드온리-에 `demon`를 설정하는 것이 있습니다. 인증 붙어 있는(SSH) 액세스 이 시나리오에서는 `svn` 클라이언트는 고유의 `svn+ssh:// URL` 형식을 사용하는; 이것은 로컬인 시큐어 조가비(SSH) 프로세스를 기동해 서버에 접속하고 나서 자기 자신을 인증합니다. 이것에는 유저는 하등의 형태의 시스템 어카운팅을 서버 위에 가지고 있을 필요가 있습니다. 인증 완료 후 SSH 프로세스가 일시적으로 내부적 `svnserve` 프로세스를 서버상에 기동해 인증되었다 유저로서 실행됩니다. 서버와 클라이언트는 암호화된 `ssh` 터널 (을)를 개입시켜 통신합니다. 이 두 개의 `svnserve`의 이용 방법은 서로 배타적인 물건은 아닌 것에 주의해 주세요; 양쪽 모두의 구조를 서버상에서 동시에 이용하는 것 하지만 할 수 있습니다. 익명 TCP/IP액세스의 설정 옵션없이 실행하면

(자)svnserve (은)는 svn 클라이언트와 세션을 확립하기 위해서 데이터를 표준 출력에 써 표준 입력으로부터 받습니다: `$ svnserve ( success ( 1 1 ( ANONYMOUS ) ( ) ) )` 이것은 직접 누군가의 도움이 되는 것은 아닙니다; svnserve 는 이 구조를 사용해inetd demon로부터 기동할 수 있게 됩니다. 그러나 svnserve (을)를 demon로서 기동하는데는 몇개의 다른 방법이 있습니다. 하나눈의 선택사항은svn를 서버머신상의 inetd의 서비스로서 등록하는 것입니다. 그렇게 해서 두어 클라이언트가 포트 3690 에 접속하려고 하면 이 번호는 인터넷 할당 번호 기관(IANA)에 의해 확보되고 있습니다. inetd 는1회 한정의 svnserve 프로세스를 기동해 클라이언트 (으)로부터의 요구를 처리합니다. 이 타입의 설정을 했을 경우 svnserve 프로세스를 root(인가 무제한의 권한을 가졌다 다른 유저)로 실행하지 않는 편이 좋을지도 모릅니다. 공개하고 있는 저장소(repository)의 소유자와 퍼미션에 의해 다른아마 새로이 만든유저인 것이 보다 바람직할 것입니다. 예를 들어 svn (이)라는 이름의 유저를 새롭게 만들어 그 유저에게 Subversion 저장소(repository)에 대한 무제한의 권리를 주어svnserve 프로세스를 그 유저로 실행하도록(듯이) 설정합니다. 물론 이 최초의 방법은 inetd(인가 그러한) demon를 가진 머신으로 만 할 수 있는 것입니다. 이것은 일반적으로 Unix상에서는 한정된 환경입니다. 다른 방법은 svnserve 를 스탠드얼론의 demon로서 실행하는 것입니다. -d 옵션 돌출하고 기동하면 svnserve 는 즉시 현재의 조가비 프로세스를 떼어내 계속 영구히 달리는 백그라운드 프로세스와 됩니다. 그리고 포트 3690으로 요구를 계속 기다립니다. `$ svnserve -d $ # svnserve is still running, but the user is returned to the prompt` 클라이언트가svnserve 프로세스에 네트워크 접속을 치면 (이것은 demon로서 실행되고 있는지 1회 한정의 처리일까에 의하지 않고) 인증 확인은 완전히 실행되지 않습니다. 서버 프로세스는 누가 그것을 실행하고 있어도 저장소(repository)에 액세스 해 클라이언트가 커밋을 발행하면 새로운 리비전이 `svn:author`속성없이 작성됩니다. 한 번 svnserve 프로그램이 실행되면 (자) 그것은 시스템상의 모든 저장소(repository)를 네트워크로부터 이용 가능하게 합니다. 바꾸어 말하면 (자) 클라이언트가 `svn://example.com/usr/local/repos/project (을)`를 체크아웃 하려고 하면 `example.com` 상에서 동작하고 있는svnserve 프로세스는 절대 패스/`usr/local/repos/project`에 있다 저장소(repository)를 보러 갑니다. 시큐리티를 확보하려면svnserve 에 -r 옵션을 건네주어 그 패스 이하의 저장소(repository)만을 공개하도록(듯이) 제한합니다: `$ svnserve -d -r /usr/local`

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

W3C XHTML 1.0 W3C CSS PHP5 POWERED

Last modified: 2005-11-03 16:40:13, Processing time: 0.0498 sec

[IE7 다운로드](#)

이제 Google에 최적화. 지금 향상된 보안 기능 친구와 사진을 같이 볼까요? Picasa 웹앨범을 사용. 무료!

[온라인 사진 앨범](#)

사용해 보세요